

AG35-CET&AG35-EUT

QuecOpen(SDK) TEE 开发指导

LTE Standard 模块系列

版本：1.0

日期：2025-06-12

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2025，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2025.

文档历史

修订记录

版本	日期	作者	变更描述
-	2023-09-23	Menson ZHANG	文档创建
1.0.0	2023-09-23	Menson ZHANG	临时版本
1.0.1	2025-03-18	Menson ZHANG	临时版本 1. 基于 QuecOpen 方案统一命名，更新文档名称。 2. 新增适用模块 AG35-EUT。 3. 新增 ql_km_initialize()和 ql_ss_initialize()的调用说明（第 3.3.1.1、3.3.3.1 章）。
1.0	2025-06-12	Menson ZHANG	受控版本

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
2 功能介绍	8
2.1. TEE 软件架构	8
2.2. 安全存储介绍	9
2.3. 加解密介绍	9
3 TEE API	10
3.1. 头文件	10
3.2. 函数概览	10
3.3. 函数详解	12
3.3.1. TrustZone 加密和解密	12
3.3.1.1. ql_km_initialize	12
3.3.1.1.1. ql_tee_error_t	12
3.3.1.2. ql_km_deinitialize	18
3.3.1.3. ql_km_gen_key	18
3.3.1.3.1. ql_km_key_args_t	19
3.3.1.3.2. ql_km_algo_t	19
3.3.1.3.3. ql_km_aes_mode_t	20
3.3.1.3.4. ql_km_ec_curve_t	20
3.3.1.3.5. ql_km_key_t	21
3.3.1.4. ql_km_import_key	21
3.3.1.4.1. ql_km_blob_t	22
3.3.1.5. ql_km_destroy_key	22
3.3.1.6. ql_km_destroy_blob	23
3.3.1.7. ql_km_operation_begin	23
3.3.1.7.1. ql_km_operation_args_t	24
3.3.1.7.2. ql_km_purpose_t	24
3.3.1.7.3. ql_km_rsa_padding_t	25
3.3.1.7.4. ql_km_digest_t	25
3.3.1.7.5. ql_km_operation_handle_t	26
3.3.1.8. ql_km_operation_update	27
3.3.1.9. ql_km_operation_finish	27
3.3.1.10. ql_km_export_key	28
3.3.1.11. ql_km_get_key_algo	29
3.3.2. 辅助简化	29
3.3.2.1. ql_aes_genkey_args	29
3.3.2.2. ql_aes_operation_args	30
3.3.2.3. ql_rsa_genkey_args	31

3.3.2.4.	ql_rsa_operation_args.....	31
3.3.2.5.	ql_ec_genkey_args	32
3.3.3.	TrustZone 安全存储.....	33
3.3.3.1.	ql_ss_initialize.....	33
3.3.3.2.	ql_ss_deinitialize	33
3.3.3.3.	ql_ss_open	34
3.3.3.4.	ql_ss_create	34
3.3.3.5.	ql_ss_close.....	35
3.3.3.6.	ql_ss_read	35
3.3.3.7.	ql_ss_write.....	36
3.3.3.8.	ql_ss_seek.....	37
3.3.3.8.1.	ql_ss_whence_t.....	37
3.3.3.9.	ql_ss_unlink.....	38
3.3.3.10.	ql_ss_trunc	38
3.3.3.11.	ql_ss_rename	39
3.3.3.12.	ql_ss_get_info.....	39
3.3.4.	宏定义.....	40
4	附录 参考文档及术语缩写	41

表格索引

表 1: 函数概览	10
表 2: 参考文档	41
表 3: 术语缩写	41

Quectel
Confidential

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen[®]方案；QuecOpen[®]是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen[®]的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen[®]方案，主要介绍 AG35-CET 和 AG35-EUT 模块的 TrustZone 安全存储设计及加解密服务 API。

Quectel
Confidential

2 功能介绍

2.1. TEE 软件架构

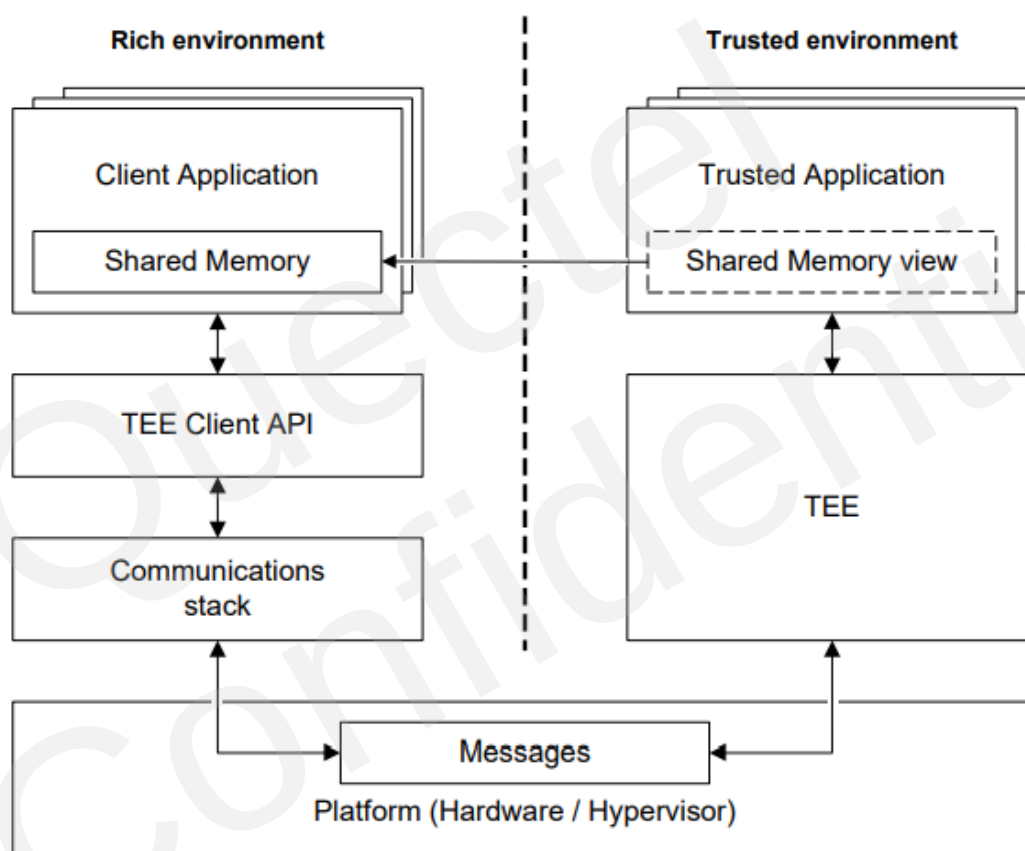


图 1: TEE 软件架构

可信执行环境（TEE）是通过创建一个可以在 TrustZone 的“安全世界”中独立运行的小型操作系统实现的，该环境可以保证算法操作不被常规操作系统干扰。

- **TA:** 即 Trusted Application，是运行在可信执行环境（TEE）下的应用程序。
- **CA:** 即 Client Application，是运行在富执行环境（REE）下的应用程序。交互流程中，由 CA 触发系统调用 TA 实现具体功能，TA 运行结束后将运行结果和数据返回给 CA。

- **Communications stack:** 通信栈，处理富执行环境和可信执行环境之间的通信。
- **TEE Client API:** 使能富执行环境中 CA 与可信执行环境中 TA 进行交互。
- **Shared Memory:** 共享内存，仅 CA 和 TA 可访问的一块安全内存。CA 和 TA 通过共享内存实现快速有效的命令和数据传输。

2.2. 安全存储介绍

可以将数据通过可信执行环境写入安全存储区域，数据被加密并储存在安全存储路径/*persist*/下，且无法直接看到文件名。

备注

在 QuecOpen SDK 中，*sample/secstor-example* 为安全存储示例程序，用户可参考其编译方式以及 API 使用方法。

2.3. 加解密介绍

AG35-CET 和 AG35-EUT 模块支持基于可信执行环境的加解密算法，包括 AES、RSA 和 ECC 等算法，并支持密钥生成、加解密、密钥导入等功能。支持的具体功能如下：

- AES 密钥生成、导入、加解密。支持 128 位和 256 位，支持 ECB、CBC、CTR 和 GCM 模式。
- ECC 密钥对生成、私钥导入、公钥导出。支持曲线 ECC-P224、ECC-P256、ECC-P384 和 ECC-P521。
- RSA 密钥对生成、私钥导入、公钥导出、加解密、签名和验签。
- RSA 支持 1024 位、2048 位、3072 位和 4096 位。
- RSA 填充算法支持 OAEP、PSS、PKCS#1V1.5。
- RSA 摘要算法支持 MD5、SHA-1、SHA2-224、SHA2-256、SHA2-384 和 SHA2-512。

备注

在 QuecOpen SDK 中，*sample/keymaster-example* 为加解密示例程序，用户可参考其编译方式以及函数使用。

3 TEE API

3.1. 头文件

TEE API 头文件为 `ql_tee_service.h`、`ql_tee_asymm_utils.h` 和 `ql_tee_symm_utils.h`，位于 SDK 包 `ql-sysroots/usr/include/` 目录下，若无特别说明，本文档所涉及头文件均在该目录下。

动态库为 `libql-tee-service.so`，位于 SDK 包 `ql-sysroots/usr/lib/` 目录下。

3.2. 函数概览

表 1: 函数概览

函数	描述
<code>ql_km_initialize()</code>	初始化加解密服务
<code>ql_km_deinitialize()</code>	去初始化加解密服务
<code>ql_km_gen_key()</code>	生成密钥
<code>ql_km_import_key()</code>	导入密钥
<code>ql_km_destroy_key()</code>	销毁密钥，释放内存
<code>ql_km_destroy_blob()</code>	销毁 BLOB 数据，释放内存
<code>ql_km_operation_begin()</code>	开始算法操作
<code>ql_km_operation_update()</code>	进行算法操作
<code>ql_km_operation_finish()</code>	进行最后一轮算法操作
<code>ql_km_export_key()</code>	导出非对称算法的公钥
<code>ql_km_get_key_algo()</code>	从 BLOB 数据中读取当前密钥的算法类别

<code>ql_aes_genkey_args()</code>	生成调用 <code>ql_km_gen_key()</code> 的 <code>key_args</code> 参数
<code>ql_aes_operation_args()</code>	生成调用 <code>ql_km_operation_begin()</code> 的参数
<code>ql_rsa_genkey_args()</code>	生成调用 <code>ql_km_gen_key()</code> 的参数
<code>ql_rsa_operation_args()</code>	生成调用 <code>ql_km_operation_begin()</code> 的参数
<code>ql_ec_genkey_args()</code>	生成调用 <code>ql_km_gen_key()</code> 的参数
<code>ql_ss_initialize()</code>	初始化安全存储功能
<code>ql_ss_deinitialize()</code>	去初始化安全存储功能
<code>ql_ss_open()</code>	打开安全存储对象
<code>ql_ss_create()</code>	创建并打开安全存储对象
<code>ql_ss_close()</code>	关闭安全存储对象
<code>ql_ss_read()</code>	从安全存储中读取数据
<code>ql_ss_write()</code>	向安全存储写入数据
<code>ql_ss_seek()</code>	移动安全存储文件的文件指针
<code>ql_ss_unlink()</code>	删除安全存储文件
<code>ql_ss_trunc()</code>	截短安全存储文件到指定的长度
<code>ql_ss_rename()</code>	重命名安全存储对象
<code>ql_ss_get_info()</code>	获取安全存储对象信息

备注

若无特别说明，本文档所述函数均不支持并发调用，且不能在相关回调函数中调用以上函数。

3.3. 函数详解

3.3.1. TrustZone 加密和解密

3.3.1.1. ql_km_initialize

该函数用于初始化加解密服务。

- 函数原型

```
ql_tee_error_t ql_km_initialize(void)
```

- 参数

无

- 返回值

QL_TEE_OK 函数执行成功
其他值 函数执行失败，详见第3.3.1.1.1章。

备注

TEE 内存存在限制，当前 SoC 配置的 TEE 的 RAM 为 2 MB，其中 TEE Core 占用接近 1 MB。加解密服务和安全存储服务隶属于不同的 TA，目前内存不支持同时加载多个 TA 应用，此接口无法与 ql_ss_initialize()同时调用，需要使用此接口时，需要先关闭安全存储 TA 应用。

3.3.1.1.1. ql_tee_error_t

错误码枚举定义如下：

```
typedef enum {
    QL_TEE_OK = 0
    QL_KM_ERROR_ROOT_OF_TRUST_ALREADY_SET = -1
    QL_KM_ERROR_UNSUPPORTED_PURPOSE = -2
    QL_KM_ERROR_INCOMPATIBLE_PURPOSE = -3
    QL_KM_ERROR_UNSUPPORTED_ALGORITHM = -4
    QL_KM_ERROR_INCOMPATIBLE_ALGORITHM = -5
    QL_KM_ERROR_UNSUPPORTED_KEY_SIZE = -6
    QL_KM_ERROR_UNSUPPORTED_BLOCK_MODE = -7
    QL_KM_ERROR_INCOMPATIBLE_BLOCK_MODE = -8
    QL_KM_ERROR_UNSUPPORTED_MAC_LENGTH = -9
    QL_KM_ERROR_UNSUPPORTED_PADDING_MODE = -10
}
```

```

QL_KM_ERROR_INCOMPATIBLE_PADDING_MODE = -11
QL_KM_ERROR_UNSUPPORTED_DIGEST = -12
QL_KM_ERROR_INCOMPATIBLE_DIGEST = -13
QL_KM_ERROR_INVALID_EXPIRATION_TIME = -14
QL_KM_ERROR_INVALID_USER_ID = -15
QL_KM_ERROR_INVALID_AUTHORIZATION_TIMEOUT = -16
QL_KM_ERROR_UNSUPPORTED_KEY_FORMAT = -17
QL_KM_ERROR_INCOMPATIBLE_KEY_FORMAT = -18
QL_KM_ERROR_UNSUPPORTED_KEY_ENCRYPTION_ALGORITHM = -19
QL_KM_ERROR_UNSUPPORTED_KEY_VERIFICATION_ALGORITHM = -20
QL_KM_ERROR_INVALID_INPUT_LENGTH = -21
QL_KM_ERROR_KEY_EXPORT_OPTIONS_INVALID = -22
QL_KM_ERROR_DELEGATION_NOT_ALLOWED = -23
QL_KM_ERROR_KEY_NOT_YET_VALID = -24
QL_KM_ERROR_KEY_EXPIRED = -25
QL_KM_ERROR_KEY_USER_NOT_AUTHENTICATED = -26
QL_KM_ERROR_OUTPUT_PARAMETER_NULL = -27
QL_KM_ERROR_INVALID_OPERATION_HANDLE = -28
QL_KM_ERROR_INSUFFICIENT_BUFFER_SPACE = -29
QL_KM_ERROR_VERIFICATION_FAILED = -30
QL_KM_ERROR_TOO_MANY_OPERATIONS = -31
QL_KM_ERROR_UNEXPECTED_NULL_POINTER = -32
QL_KM_ERROR_INVALID_KEY_BLOB = -33
QL_KM_ERROR_IMPORTED_KEY_NOT_ENCRYPTED = -34
QL_KM_ERROR_IMPORTED_KEY_DECRYPTION_FAILED = -35
QL_KM_ERROR_IMPORTED_KEY_NOT_SIGNED = -36
QL_KM_ERROR_IMPORTED_KEY_VERIFICATION_FAILED = -37
QL_KM_ERROR_INVALID_ARGUMENT = -38
QL_KM_ERROR_UNSUPPORTED_TAG = -39
QL_KM_ERROR_INVALID_TAG = -40
QL_KM_ERROR_MEMORY_ALLOCATION_FAILED = -41
QL_KM_ERROR_IMPORT_PARAMETER_MISMATCH = -44
QL_KM_ERROR_SECURE_HW_ACCESS_DENIED = -45
QL_KM_ERROR_OPERATION_CANCELLED = -46
QL_KM_ERROR_CONCURRENT_ACCESS_CONFLICT = -47
QL_KM_ERROR_SECURE_HW_BUSY = -48
QL_KM_ERROR_SECURE_HW_COMMUNICATION_FAILED = -49
QL_KM_ERROR_UNSUPPORTED_EC_FIELD = -50
QL_KM_ERROR_MISSING_NONCE = -51
QL_KM_ERROR_INVALID_NONCE = -52
QL_KM_ERROR_MISSING_MAC_LENGTH = -53
QL_KM_ERROR_KEY_RATE_LIMIT_EXCEEDED = -54
QL_KM_ERROR_CALLER_NONCE_PROHIBITED = -55
QL_KM_ERROR_KEY_MAX_OPS_EXCEEDED = -56

```

```

QL_KM_ERROR_INVALID_MAC_LENGTH = -57
QL_KM_ERROR_MISSING_MIN_MAC_LENGTH = -58
QL_KM_ERROR_UNSUPPORTED_MIN_MAC_LENGTH = -59
QL_KM_ERROR_UNSUPPORTED_KDF = -60
QL_KM_ERROR_UNSUPPORTED_EC_CURVE = -61
QL_KM_ERROR_KEY_REQUIRES_UPGRADE = -62
QL_KM_ERROR_ATTESTATION_CHALLENGE_MISSING = -63
QL_KM_ERROR_KEYMASTER_NOT_CONFIGURED = -64
QL_KM_ERROR_ATTESTATION_APPLICATION_ID_MISSING = -65
QL_KM_ERROR_UNIMPLEMENTED = -100
QL_KM_ERROR_VERSION_MISMATCH = -101
QL_KM_ERROR_UNKNOWN_ERROR = -1000
QL_SS_ERROR_GENERIC = -200
QL_SS_ERROR_ACCESS_DENIED = -201
QL_SS_ERROR_CANCEL = -202
QL_SS_ERROR_ACCESS_CONFLICT = -203
QL_SS_ERROR_EXCESS_DATA = -204
QL_SS_ERROR_BAD_FORMAT = -205
QL_SS_ERROR_BAD_PARAMETERS = -206
QL_SS_ERROR_BAD_STATE = -207
QL_SS_ERROR_ITEM_NOT_FOUND = -208
QL_SS_ERROR_NOT_IMPLEMENTED = -209
QL_SS_ERROR_NOT_SUPPORTED = -210
QL_SS_ERROR_NO_DATA = -211
QL_SS_ERROR_OUT_OF_MEMORY = -212
QL_SS_ERROR_BUSY = -213
QL_SS_ERROR_COMMUNICATION = -214
QL_SS_ERROR_SECURITY = -215
QL_SS_ERROR_SHORT_BUFFER = -216
QL_SS_ERROR_EXTERNAL_CANCEL = -217
QL_SS_ERROR_TARGET_DEAD = -2000
QL_SS_ERROR_UNKNOWN_ERROR = -2001
} ql_tee_error_t;

```

● 成员

成员	描述
QL_TEE_OK	成功
QL_KM_ERROR_ROOT_OF_TRUST_ALREADY_SET	可信根已设置
QL_KM_ERROR_UNSUPPORTED_PURPOSE	操作不支持
QL_KM_ERROR_INCOMPATIBLE_PURPOSE	操作不兼容

QL_KM_ERROR_UNSUPPORTED_ALGORITHM	算法不支持
QL_KM_ERROR_INCOMPATIBLE_ALGORITHM	算法不兼容
QL_KM_ERROR_UNSUPPORTED_KEY_SIZE	密钥大小不支持
QL_KM_ERROR_UNSUPPORTED_BLOCK_MODE	块模式不支持
QL_KM_ERROR_INCOMPATIBLE_BLOCK_MODE	块模式不兼容
QL_KM_ERROR_UNSUPPORTED_MAC_LENGTH	MAC 长度不支持
QL_KM_ERROR_UNSUPPORTED_PADDING_MODE	填充模式不支持
QL_KM_ERROR_INCOMPATIBLE_PADDING_MODE	填充模式不兼容
QL_KM_ERROR_UNSUPPORTED_DIGEST	摘要模式不支持
QL_KM_ERROR_INCOMPATIBLE_DIGEST	摘要模式不兼容
QL_KM_ERROR_INVALID_EXPIRATION_TIME	无效的到期时间
QL_KM_ERROR_INVALID_USER_ID	无效的用户 ID
QL_KM_ERROR_INVALID_AUTHORIZATION_TIMEOUT	无效的授权超时
QL_KM_ERROR_UNSUPPORTED_KEY_FORMAT	密钥格式不支持
QL_KM_ERROR_INCOMPATIBLE_KEY_FORMAT	密钥格式不兼容
QL_KM_ERROR_UNSUPPORTED_KEY_ENCRYPTION_ALGORITHM	不支持的加密算法
QL_KM_ERROR_UNSUPPORTED_KEY_VERIFICATION_ALGORITHM	不支持的验证算法
QL_KM_ERROR_INVALID_INPUT_LENGTH	输入长度无效
QL_KM_ERROR_KEY_EXPORT_OPTIONS_INVALID	密钥导出选项无效
QL_KM_ERROR_DELEGATION_NOT_ALLOWED	授权不允许
QL_KM_ERROR_KEY_NOT_YET_VALID	密钥未生效
QL_KM_ERROR_KEY_EXPIRED	密钥过期
QL_KM_ERROR_KEY_USER_NOT_AUTHENTICATED	用户未通过身份验证
QL_KM_ERROR_OUTPUT_PARAMETER_NULL	输出参数空
QL_KM_ERROR_INVALID_OPERATION_HANDLE	无效的操作句柄
QL_KM_ERROR_INSUFFICIENT_BUFFER_SPACE	缓冲空间不足

QL_KM_ERROR_VERIFICATION_FAILED	验证失败
QL_KM_ERROR_TOO_MANY_OPERATIONS	过多操作
QL_KM_ERROR_UNEXPECTED_NULL_POINTER	意外的空指针
QL_KM_ERROR_INVALID_KEY_BLOB	无效的密钥块
QL_KM_ERROR_IMPORTED_KEY_NOT_ENCRYPTED	导入密钥未加密
QL_KM_ERROR_IMPORTED_KEY_DECRYPTION_FAILED	导入密钥解密失败
QL_KM_ERROR_IMPORTED_KEY_NOT_SIGNED	导入密钥未签名
QL_KM_ERROR_IMPORTED_KEY_VERIFICATION_FAILED	导入密钥验证失败
QL_KM_ERROR_INVALID_ARGUMENT	无效的参数
QL_KM_ERROR_UNSUPPORTED_TAG	不支持的标签
QL_KM_ERROR_INVALID_TAG	无效的标签
QL_KM_ERROR_MEMORY_ALLOCATION_FAILED	内存分配失败
QL_KM_ERROR_IMPORT_PARAMETER_MISMATCH	导入参数不匹配
QL_KM_ERROR_SECURE_HW_ACCESS_DENIED	安全硬件访问被拒绝
QL_KM_ERROR_OPERATION_CANCELLED	操作被取消
QL_KM_ERROR_CONCURRENT_ACCESS_CONFLICT	并发访问冲突
QL_KM_ERROR_SECURE_HW_BUSY	安全硬件繁忙
QL_KM_ERROR_SECURE_HW_COMMUNICATION_FAILED	安全硬件通信失败
QL_KM_ERROR_UNSUPPORTED_EC_FIELD	不支持的 EC 字段
QL_KM_ERROR_MISSING_NONCE	缺少随机数
QL_KM_ERROR_INVALID_NONCE	无效的随机数
QL_KM_ERROR_MISSING_MAC_LENGTH	缺少 MAC 长度
QL_KM_ERROR_KEY_RATE_LIMIT_EXCEEDED	超出密钥速率限制
QL_KM_ERROR_CALLER_NONCE_PROHIBITED	禁止调用随机数
QL_KM_ERROR_KEY_MAX_OPS_EXCEEDED	已超过密钥最大操作数
QL_KM_ERROR_INVALID_MAC_LENGTH	无效的 MAC 长度
QL_KM_ERROR_MISSING_MIN_MAC_LENGTH	缺少最小 MAC 长度

QL_KM_ERROR_UNSUPPORTED_MIN_MAC_LENGTH	不支持的最小 MAC 长度
QL_KM_ERROR_UNSUPPORTED_KDF	不支持的密钥派生
QL_KM_ERROR_UNSUPPORTED_EC_CURVE	不支持的 ECC 曲线
QL_KM_ERROR_KEY_REQUIRES_UPGRADE	密钥需要升级
QL_KM_ERROR_ATTESTATION_CHALLENGE_MISSING	证明缺失
QL_KM_ERROR_KEYMASTER_NOT_CONFIGURED	未配置
QL_KM_ERROR_ATTESTATION_APPLICATION_ID_MISSING	缺少证明应用 ID
QL_KM_ERROR_UNIMPLEMENTED	功能未实现
QL_KM_ERROR_VERSION_MISMATCH	版本不匹配
QL_KM_ERROR_UNKNOWN_ERROR	未知错误
QL_SS_ERROR_GENERIC	通用错误码
QL_SS_ERROR_ACCESS_DENIED	拒绝访问
QL_SS_ERROR_CANCEL	操作取消
QL_SS_ERROR_ACCESS_CONFLICT	访问冲突
QL_SS_ERROR_EXCESS_DATA	数据超大
QL_SS_ERROR_BAD_FORMAT	错误格式
QL_SS_ERROR_BAD_PARAMETERS	错误参数
QL_SS_ERROR_BAD_STATE	错误状态
QL_SS_ERROR_ITEM_NOT_FOUND	资源不存在
QL_SS_ERROR_NOT_IMPLEMENTED	未实现
QL_SS_ERROR_NOT_SUPPORTED	不支持
QL_SS_ERROR_NO_DATA	无此数据
QL_SS_ERROR_OUT_OF_MEMORY	内存耗尽
QL_SS_ERROR_BUSY	遇忙
QL_SS_ERROR_COMMUNICATION	通信失败
QL_SS_ERROR_SECURITY	安全验证失败
QL_SS_ERROR_SHORT_BUFFER	Buffer 过小

QL_SS_ERROR_EXTERNAL_CANCEL	外部取消
QL_SS_ERROR_TARGET_DEAD	对象已关闭
QL_SS_ERROR_UNKNOWN_ERROR	未知错误

3.3.1.2. ql_km_deinitialize

该函数用于去初始化加解密服务。

- 函数原型

```
void ql_km_deinitialize(void)
```

- 参数

无

- 返回值

无

3.3.1.3. ql_km_gen_key

该函数用于生成密钥。

- 函数原型

```
ql_tee_error_t ql_km_gen_key(const ql_km_key_args_t *key_args, ql_km_key_t *key)
```

- 参数

key_args:

[In] 待生成的密钥的参数。如算法、密钥长度等；详见第3.3.1.3.1章。

key:

[Out] 生成的密钥。内存在函数调用过程中分配；详见第3.3.1.3.5章。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.1.3.1. ql_km_key_args_t

密钥参数结构体定义如下：

```
typedef struct {
    ql_km_algo_t algo;
    union {
        struct {
            ql_km_aes_mode_t mode;
            uint32_t bits;
            uint32_t min_mac_len;
        } aes_args;
        struct {
            uint64_t exponent;
            uint32_t bits;
        } rsa_args;
        struct {
            ql_km_ec_curve_t curve;
        } ec_args;
    };
} ql_km_key_args_t;
```

● 参数

类型	参数	描述
<i>ql_km_algo_t</i>	<i>algo</i>	算法类型；详见第3.3.1.3.2章。
<i>ql_km_aes_mode_t</i>	<i>mode</i>	AES 密钥模式；详见第3.3.1.3.3章。
<i>uint32_t</i>	<i>bits</i>	密钥位数
<i>uint32_t</i>	<i>min_mac_len</i>	GCM 模式中 MAC 的最小长度
<i>uint64_t</i>	<i>exponent</i>	RSA 密钥的指数
<i>ql_km_ec_curve_t</i>	<i>curve</i>	ECC 曲线；详见第3.3.1.3.4章。

3.3.1.3.2. ql_km_algo_t

算法类型枚举定义如下：

```
typedef enum {
    QL_KM_ALGO_RSA = 1,
    QL_KM_ALGO_EC = 3,
    QL_KM_ALGO_AES = 32,
} ql_km_algo_t;
```

● 成员

成员	描述
QL_KM_ALGO_RSA	RSA 算法
QL_KM_ALGO_EC	ECC 算法
QL_KM_ALGO_AES	AES 算法

3.3.1.3.3. ql_km_aes_mode_t

AES 的密钥模式枚举定义如下：

```
typedef enum {
    QL_KM_MODE_ECB = 1,
    QL_KM_MODE_CBC = 2,
    QL_KM_MODE_CTR = 3,
    QL_KM_MODE_GCM = 32,
} ql_km_aes_mode_t;
```

● 成员

成员	描述
QL_KM_MODE_ECB	ECB 模式
QL_KM_MODE_CBC	CBC 模式
QL_KM_MODE_CTR	CTR 模式
QL_KM_MODE_GCM	GCM 模式

3.3.1.3.4. ql_km_ec_curve_t

ECC 曲线枚举定义如下：

```
typedef enum {
    QL_KM_EC_CURVE_P_224 = 0,
    QL_KM_EC_CURVE_P_256 = 1,
    QL_KM_EC_CURVE_P_384 = 2,
    QL_KM_EC_CURVE_P_521 = 3,
} ql_km_ec_curve_t;
```

● 成员

成员	描述
<code>QL_KM_EC_CURVE_P_224</code>	ECC-P224 曲线
<code>QL_KM_EC_CURVE_P_256</code>	ECC-P256 曲线
<code>QL_KM_EC_CURVE_P_384</code>	ECC-P384 曲线
<code>QL_KM_EC_CURVE_P_521</code>	ECC-P521 曲线

3.3.1.3.5. ql_km_key_t

密钥结构体定义如下：

```
typedef struct {
    uint8_t *key_blob;
    uint32_t key_blob_size;
} ql_km_key_t;
```

● 参数

类型	参数	描述
<code>uint8_t*</code>	<code>key_blob</code>	密钥
<code>uint32_t</code>	<code>key_blob_size</code>	密钥长度

3.3.1.4. ql_km_import_key

该函数用于导入密钥。

● 函数原型

```
ql_tee_error_t ql_km_import_key(const ql_km_key_args_t *key_args, const ql_km_blob_t *rawkey, ql_km_key_t *key)
```

● 参数

`key_args`:

[In] 待导入的密钥的参数。如算法、密钥长度等；详见第3.3.1.3.1章。

`rawkey`:

[In] 待导入的 BLOB 数据；详见第3.3.1.4.1章。

key:

[Out] 生成的 BLOB 数据。内存在函数调用过程中分配；详见第 3.3.1.3.5 章。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第 3.3.1.1.1 章。

3.3.1.4.1. ql_km_blob_t

BLOB 数据结构体定义如下：

```
typedef struct {
    uint8_t *data;
    uint32_t data_size;
} ql_km_blob_t;
```

- 参数

类型	参数	描述
uint8_t*	data	数据
uint32_t	data_size	数据长度

3.3.1.5. ql_km_destroy_key

该函数用于销毁密钥，释放内存。

- 函数原型

```
ql_tee_error_t ql_km_destroy_key(ql_km_key_t *key)
```

- 参数

key:

[Out] 待销毁的密钥。内存在函数调用过程中分配；详见第 3.3.1.3.5 章。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第 3.3.1.1.1 章。

3.3.1.6. ql_km_destroy_blob

该函数用于销毁 BLOB 数据，释放内存。

- 函数原型

```
ql_tee_error_t ql_km_destroy_blob(ql_km_blob_t *blob)
```

- 参数

blob:

[Out] 待销毁的 BLOB 数据；详见第 3.3.1.4.1 章。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第 3.3.1.1.1 章。

3.3.1.7. ql_km_operation_begin

该函数用于开始一个算法操作。如加密、解密、签名或验签等。

- 函数原型

```
ql_tee_error_t ql_km_operation_begin(const ql_km_key_t *key, ql_km_operation_args_t *op_args, ql_km_operation_handle_t *op_handle)
```

- 参数

key:

[In] 算法操作所需要的密钥；详见第 3.3.1.3.5 章。

op_args:

[In] 算法操作所需要的参数；详见第 3.3.1.7.1 章。

op_handle:

[Out] 操作句柄；详见第 3.3.1.7.5 章。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第 3.3.1.1.1 章。

3.3.1.7.1. ql_km_operation_args_t

算法操作所需参数结构体定义如下：

```
typedef struct {
    ql_km_algo_t algo;
    ql_km_purpose_t purpose;
    union {
        struct {
            ql_km_aes_mode_t mode;
            ql_km_blob_t iv;
        } aes_args;
        struct {
            ql_km_rsa_padding_t padding;
            ql_km_digest_t digest;
        } rsa_args;
    };
} ql_km_operation_args_t;
```

● 参数

类型	参数	描述
<i>ql_km_algo_t</i>	<i>algo</i>	算法类型；详见第3.3.1.3.2章。
<i>ql_km_purpose_t</i>	<i>purpose</i>	操作类别；详见第3.3.1.7.2章。
<i>ql_km_aes_mode_t</i>	<i>mode</i>	AES 模式；详见第3.3.1.3.3章。
<i>ql_km_blob_t</i>	<i>iv</i>	偏移量；详见第3.3.1.4.1章。
<i>ql_km_rsa_padding_t</i>	<i>padding</i>	填充算法；详见第3.3.1.7.3章。
<i>ql_km_digest_t</i>	<i>digest</i>	摘要算法；详见第3.3.1.7.4章。

3.3.1.7.2. ql_km_purpose_t

操作类别枚举定义如下：

```
typedef enum {
    QL_KM_PURPOSE_ENCRYPT = 0,
    QL_KM_PURPOSE_DECRYPT = 1,
    QL_KM_PURPOSE_SIGN = 2,
    QL_KM_PURPOSE_VERIFY = 3,
} ql_km_purpose_t;
```

- 成员

成员	描述
<code>QL_KM_PURPOSE_ENCRYPT</code>	加密
<code>QL_KM_PURPOSE_DECRYPT</code>	解密
<code>QL_KM_PURPOSE_SIGN</code>	签名
<code>QL_KM_PURPOSE_VERIFY</code>	验签

3.3.1.7.3. ql_km_rsa_padding_t

RSA 的填充算法枚举定义如下：

```
typedef enum {
    QL_KM_PAD_RSA_NONE = 1,
    QL_KM_PAD_RSA_OAEP = 2,
    QL_KM_PAD_RSA_PSS = 3,
    QL_KM_PAD_RSA_PKCS1_1_5_ENCRYPT = 4,
    QL_KM_PAD_RSA_PKCS1_1_5_SIGN = 5,
} ql_km_rsa_padding_t;
```

- 成员

成员	描述
<code>QL_KM_PAD_RSA_NONE</code>	无填充
<code>QL_KM_PAD_RSA_OAEP</code>	OAEP
<code>QL_KM_PAD_RSA_PSS</code>	RSA-PSS
<code>QL_KM_PAD_RSA_PKCS1_1_5_ENCRYPT</code>	PKCS#1v1.5
<code>QL_KM_PAD_RSA_PKCS1_1_5_SIGN</code>	PKCS#1v1.5

3.3.1.7.4. ql_km_digest_t

摘要算法枚举定义如下：

```
typedef enum {
    QL_KM_DIGEST_NONE = 0,
    QL_KM_DIGEST_MD5 = 1,
    QL_KM_DIGEST_SHA1 = 2,
    QL_KM_DIGEST_SHA_2_224 = 3,
    QL_KM_DIGEST_SHA_2_256 = 4,
```

```
QL_KM_DIGEST_SHA_2_384 = 5,
QL_KM_DIGEST_SHA_2_512 = 6,
} ql_km_digest_t;
```

● 成员

成员	描述
<i>QL_KM_DIGEST_NONE</i>	无相应算法
<i>QL_KM_DIGEST_MD5</i>	MD5 算法
<i>QL_KM_DIGEST_SHA1</i>	SHA-1 算法
<i>QL_KM_DIGEST_SHA_2_224</i>	SHA-224 算法
<i>QL_KM_DIGEST_SHA_2_256</i>	SHA-256 算法
<i>QL_KM_DIGEST_SHA_2_384</i>	SHA-384 算法
<i>QL_KM_DIGEST_SHA_2_512</i>	SHA-512 算法

3.3.1.7.5. ql_km_operation_handle_t

操作句柄结构体定义如下：

```
typedef struct {
    ql_km_algo_t algo;
    ql_km_purpose_t purpose;
    uint64_t handle;
} ql_km_operation_handle_t;
```

● 参数

类型	参数	描述
<i>ql_km_algo_t</i>	<i>algo</i>	算法类型；详见第3.3.1.3.2章。
<i>ql_km_purpose_t</i>	<i>purpose</i>	操作类别；详见第3.3.1.7.2章。
<i>uint64_t</i>	<i>handle</i>	句柄

3.3.1.8. ql_km_operation_update

该函数用于进行算法操作。如加密、解密、签名或验签。

● 函数原型

```
ql_tee_error_t ql_km_operation_update(ql_km_operation_handle_t *op_handle, uint8_t *input,
uint32_t input_size, ql_km_blob_t *output, uint32_t *consumed)
```

● 参数

op_handle:

[In] 操作句柄。详见第3.3.1.7.5章。

input:

[In] 待处理数据。

input_size:

[In] 待处理数据的大小。

output:

[Out] 操作结果。内存在函数调用过程中分配；详见第3.3.1.4.1章。

consumed:

[Out] 本次算法操作实际输入数据量。单位：字节。

● 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.1.9. ql_km_operation_finish

该函数用于进行最后一轮算法操作。如加密、解密、签名或验签。

● 函数原型

```
ql_tee_error_t ql_km_operation_finish(ql_km_operation_handle_t *op_handle, uint8_t *input,
uint32_t input_size, ql_km_blob_t *output, uint8_t *signature, uint32_t sig_size)
```

● 参数

op_handle:

[In] 操作句柄；详见第3.3.1.7.5章。

input:

[In] 待处理数据。

input_size:

[In] 待处理数据长度。

output:

[Out] 操作结果。内存在函数调用过程中分配；详见第3.3.1.4.1章。

signature:

[In] 数据的签名。

sig_size:

[In] 签名的长度。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

备注

signature 和 *sig_size* 为可选参数。若要进行加密、解密或签名操作，不必配置 *signature* 和 *sig_size* 参数。若要进行验签操作，则必须配置 *signature* 和 *sig_size* 参数。

3.3.1.10. ql_km_export_key

该函数用于导出非对称算法的公钥。

- 函数原型

```
ql_tee_error_t ql_km_export_key(ql_km_key_t *keypair, ql_km_blob_t *pubkey)
```

- 参数

keypair:

[In] 非对称密钥对；详见第3.3.1.3.5章。

pubkey:

[Out] BLOB 数据。导出的公钥，内存在函数调用过程中分配；详见第3.3.1.4.1章。

- 返回值

`QL_TEE_OK` 函数执行成功
其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.1.11. ql_km_get_key_algo

该函数用于从 BLOB 数据中读取当前密钥的算法类别。

- 函数原型

```
ql_tee_error_t ql_km_get_key_algo(ql_km_key_t *keyblob, ql_km_algo_t *algo)
```

- 参数

keyblob:
[In] 密钥；详见第3.3.1.3.5章。

algo:
[Out] 算法类型；详见第3.3.1.3.2章。

- 返回值

`QL_TEE_OK` 函数执行成功
其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.2. 辅助简化

为了简化传入参数设置，方便用户调用上述 API，移远通信提供以下 API 以简化 AES，RSA 和 ECC 算法的操作。

3.3.2.1. ql_aes_genkey_args

该函数用于生成调用 `ql_km_gen_key()` 的 *key_args* 参数。

- 函数原型

```
ql_tee_error_t ql_aes_genkey_args(ql_km_aes_mode_t mode, uint32_t bits, ql_km_key_args_t *args, uint32_t min_mac_len)
```

- 参数

mode:

[In] AES 密钥模式；详见第3.3.1.3.3章。

bits:

[In] 密钥位数。

args:

[Out] 密钥参数。如算法、密钥长度等，不分配内存；详见第3.3.1.3.1章。

min_mac_len:

[In] GCM 模式中 MAC 的最小长度。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

备注

min_mac_len 为可选参数。若在 CBC、CTR 或 ECB 模式下使用，不必配置该参数。若在 GCM 模式下使用，则必须配置该参数。

3.3.2.2. ql_aes_operation_args

该函数用于生成调用 *ql_km_operation_begin()* 的参数。

- 函数原型

```
ql_tee_error_t ql_aes_operation_args(ql_km_purpose_t purpose, ql_km_aes_mode_t mode,
                                     ql_km_blob_t *iv, ql_km_operation_args_t *args)
```

- 参数

purpose:

[In] 操作类别；详见第3.3.1.7.2章。

mode:

[In] 算法模式；详见第3.3.1.3.3章。

iv:

[In] 偏移量。当 *purpose* 为解密时输入；详见第3.3.1.4.1章。

args:

[Out] 算法所需参数，用来调用 *ql_km_operation_begin()*。不分配内存；详见第3.3.1.7.1章。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.2.3. ql_rsa_genkey_args

该函数用于生成调用 *ql_km_gen_key()*的参数。

- 函数原型

```
ql_tee_error_t ql_rsa_genkey_args(uint64_t exponent, uint32_t bits, ql_km_key_args_t *args)
```

- 参数

exponent:

[In] RSA 密钥的指数。

bits:

[In] 密钥的位数。

args:

[Out] 算法所需参数，用来调用 *ql_km_gen_key()*。不分配内存；详见第3.3.1.7.1章。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.2.4. ql_rsa_operation_args

该函数用于生成调用 *ql_km_operation_begin()*的参数。

- 函数原型

```
ql_tee_error_t ql_rsa_operation_args(ql_km_purpose_t purpose, ql_km_rsa_padding_t padding,
                                     ql_km_digest_t digest, ql_km_operation_args_t *args)
```

- 参数

purpose:

[In] 操作类别；详见第3.3.1.7.2章。

padding:

[In] 填充算法；详见第3.3.1.7.3章。

digest:

[In] 摘要算法；详见第3.3.1.7.4章。

args:

[In] 算法所需参数，用来调用 *ql_km_operation_begin()*。不分配内存；详见第3.3.1.7.1章。

- 返回值

QL_TEE_OK 函数执行成功。

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.2.5. ql_ec_genkey_args

该函数用于生成调用 *ql_km_gen_key()* 的参数。

- 函数原型

```
ql_tee_error_t ql_ec_genkey_args(ql_km_ec_curve_t curve, ql_km_key_args_t *args)
```

- 参数

curve:

[In] ECC 曲线；详见第3.3.1.3.4章。

args:

[Out] 算法所需参数，用来调用 *ql_km_gen_key()*。不分配内存；详见第3.3.1.3.1章。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3. TrustZone 安全存储

TrustZone 安全存储将安全数据存储在 */persist* 路径下，客户可根据需要对该目录进行备份和还原。

3.3.3.1. ql_ss_initialize

该函数用于初始化安全存储功能。在调用其他函数前，先调用本函数初始化安全存储服务。

- 函数原型

```
ql_tee_error_t ql_ss_initialize(void)
```

- 参数

无

- 返回值

QL_TEE_OK 函数执行成功
其他值 函数执行失败，详见第3.3.1.1.1章。

备注

TEE 内存存在限制，当前 SoC 配置的 TEE 的 RAM 为 2 MB，其中 TEE Core 占用接近 1 MB。加解密服务和安全存储服务隶属于不同的 TA，目前内存不支持同时加载多个 TA 应用，此接口无法与 *ql_km_initialize()* 同时调用，需要使用此接口时，需要先关闭加解密服务 TA 应用。

3.3.3.2. ql_ss_deinitialize

该函数用于去初始化安全存储功能。若不再使用安全存储功能，调用该函数以释放资源。

- 函数原型

```
void ql_ss_deinitialize(void)
```

- 参数

无

- 返回值

无

3.3.3.3. ql_ss_open

该函数用于打开安全存储对象。

- 函数原型

```
ql_tee_error_t ql_ss_open(const void *id, uint32_t id_size, uint32_t *object)
```

- 参数

id:

[In] 安全存储中的对象名称。

id_size:

[In] *id* 的大小。单位：字节。

object:

[Out] 安全存储对象句柄。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.4. ql_ss_create

该函数用于创建并打开安全存储对象。指向安全存储文件头。

- 函数原型

```
ql_tee_error_t ql_ss_create(const void *id, uint32_t id_size, void *data ,uint32_t data_size,
uint32_t *object, bool overwrite)
```

- 参数

id:

[In] 安全存储中的对象名称。

id_size:

[In] *id* 的大小。单位：字节。

data:

[In] 需要写入安全存储的数据。

data_size:

[In] 需要写入安全存储的数据大小。单位：字节。

object:

[Out] 安全存储对象句柄。

overwrite:

[In] 是否覆盖同名对象。

TRUE 覆盖

FALSE 不覆盖

● 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.5. ql_ss_close

该函数用于关闭安全存储对象。

● 函数原型

```
ql_tee_error_t ql_ss_close(uint32_t object)
```

● 参数

object:

[In] 安全存储对象句柄。

● 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.6. ql_ss_read

该函数用于从安全存储中读取数据。

● 函数原型

```
ql_tee_error_t ql_ss_read(uint32_t object, void *data, uint32_t data_size, uint32_t *count)
```

- 参数

object:

[In] 安全存储对象句柄。

data:

[Out] 用于存储数据的地址。

data_size:

[In] 需要读取的数据大小。单位：字节。

count:

[Out] 实际读取的数据大小。单位：字节。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.7. ql_ss_write

该函数用于向安全存储写入数据。

- 函数原型

```
ql_tee_error_t ql_ss_write(uint32_t object, void *data, uint32_t data_size)
```

- 参数

object:

[In] 安全存储对象句柄。

data:

[In] 需要写入的数据。

data_size:

[In] 需要写入的数据大小。单位：字节。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.8. ql_ss_seek

该函数用于移动安全存储文件的文件指针。

● 函数原型

```
ql_tee_error_t ql_ss_seek(uint32_t object, int32_t offset, ql_ss_whence_t whence)
```

● 参数

object:

[In] 安全存储对象句柄。

offset:

[In] 偏移量。

whence:

[In] 移动文件指针的起始位置；详见第3.3.3.8.1章。

● 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.8.1. ql_ss_whence_t

安全存储移位时相对位置枚举定义如下：

```
typedef enum {
    QL_SS_SEEK_SET = 0,
    QL_SS_SEEK_CUR = 1,
    QL_SS_SEEK_END = 2
} ql_ss_whence_t;
```

● 成员

成员	描述
QL_SS_SEEK_SET	安全存储文件头
QL_SS_SEEK_CUR	安全存储文件指针当前位置
QL_SS_SEEK_END	安全存储文件尾

3.3.3.9. ql_ss_unlink

该函数用于删除安全存储文件。

- 函数原型

```
ql_tee_error_t ql_ss_unlink(uint32_t object)
```

- 参数

object:

[In] 安全存储对象句柄。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.10. ql_ss_trunc

该函数用于截短安全存储文件到指定的长度。

- 函数原型

```
ql_tee_error_t ql_ss_trunc(uint32_t object, uint32_t len)
```

- 参数

object:

[In] 安全存储对象句柄。

len:

[In] 截取的目标长度。单位：字节。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.11. ql_ss_rename

该函数用于重命名安全存储对象。

- 函数原型

```
ql_tee_error_t ql_ss_rename(uint32_t object, const void *id, uint32_t id_size)
```

- 参数

object:

[In] 安全存储对象句柄。

id:

[In] 新的对象名。

id_size:

[In] 对象名长度。单位：字节。

- 返回值

QL_TEE_OK 函数执行成功

其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.3.12. ql_ss_get_info

该函数用于获取安全存储对象信息，包括文件大小和当前文件指针的位置。

- 函数原型

```
ql_tee_error_t ql_ss_get_info(uint32_t object, uint32_t *obj_size, uint32_t *cur_pos)
```

- 参数

object:

[In] 安全存储对象句柄。

obj_size:

[Out] 对象的大小。

cur_pos:

[Out] 当前文件指针的位置。

- 返回值

QL_TEE_OK 函数执行成功
其他值 函数执行失败，详见第3.3.1.1.1章。

3.3.4. 宏定义

宏定义	描述
#define OPERATION_BUF_MAX_SIZE 4096	单次操作最大字节数
#define SINGLE_OBJ_MAX_SIZE (512 * 1024)	安全存储单个对象最大字节数

4 附录 参考文档及术语缩写

表 2：参考文档

文档名称
[1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导

表 3：术语缩写

缩写	英文全称	中文全称
AES	Advanced Encryption Standard	高级加密标准
API	Application Programming Interface	应用程序编程接口
BLOB	Binary Large Object	二进制型大对象
CBC	Cipher Block Chaining	分组密码工作模式
CTX	Context	上下文
ECB	Electronic Codebook	电码本
ECC	Elliptic Curve Cryptography	椭圆加密算法
GCM	Galois/Counter Mode	伽罗瓦/计数器模式
IoV	Internet of Vehicles	车联网
IV	Initialization Vector	初始化向量
MD5	MD5 Message-Digest Algorithm	MD5 信息摘要算法
OAEP	Optimal Asymmetric Encryption Padding	最优非对称加密填充
PKCS	Public Key Cryptography Standards	公钥密码学标准
PSS	Probability Signature Scheme	概率签名方案

RSA	Rivest, Shamir, Adleman	RSA 算法
SS	Secure Storage	安全存储
TEE	Trusted Execution Environment	可信执行环境

Quectel
Confidential